Name: Wilbert Chandra

NIM: 2602113666

PROBLEM 1

What is Optimal BST?

An Optimal Binary Search Tree (OBST) is a unique binary search tree designed to minimize the time it takes to find a key. Each node in an OBST has a weight, representing the likelihood of that key being searched, with the total weight of all nodes equaling 1.0. The expected search time for a node is calculated as its depth (distance from the root) multiplied by its weight, plus the expected search time of its children. To build an OBST, we start with a sorted list of keys and their probabilities, create a table of expected search times for all possible sub-trees using dynamic programming, and then use this table to construct the OBST. The construction process has a time complexity of $O(n\text{^}3)$, which can be optimized to $O(n\text{^}2)$, and once built, the search time is $O(log\ n)$, like a regular binary search tree. OBSTs are particularly useful in scenarios where keys have varying search probabilities, enhancing efficiency in databases, compilers, and other software applications.

Why do we need it to solve a problem?

The Optimal Binary Search Tree (OBST) is a tool that helps us find things quickly in a set of items, each with its own known likelihood of being searched. Here's why it's useful:

1. Efficiency: OBST places items that are often searched near the top of the tree, reducing the steps needed to find them. This boosts performance in scenarios where searches are common.
2. Adaptability: OBST can be built for any set of items and their search probabilities, making it versatile for various problems.
3. Dynamic Programming Insight: Building an OBST is a classic case of dynamic programming, a method that solves complex problems by breaking them into simpler ones. This offers a practical understanding of how dynamic programming operates.
4. Use Cases: OBST is used in many areas like database systems for indexing (where records have different access probabilities), compilers (where different variables and constants have different access probabilities), and other software where data is accessed at different rates.

Give an example of a problem solved with Optimal BST!

Problem Example: Dictionary Search

We are designing a spell-checking software for a word processor. The software needs to quickly search for words in a dictionary to check their correctness. Given the following words and their probabilities of being searched:

("apple", 0.2)

("banana", 0.1)

("cherry", 0.4)

("date", 0.1)

("fig", 0.2)

The probabilities represent how frequently each word is likely to be searched. Our **goal is to build a BST that minimizes the average search time based on these probabilities**.

Solution Using Optimal BST Algorithm

1. Construct the Cost and Root Matrices:

   We need to construct two matrices:

   $cost[i][j]$: Represents the cost of the optimal BST for the subarray from i to j.

   $root[i][j]$: Stores the root of the subtree that spans from i to j.

2. Initialize the Matrices:

For each word i initialize $cost[i][i]$ to the probability of the word being searched, because the cost to access a single node is simply its probability.

```
cost[i][i] = probability[i]
```

For ranges where $i > j$ (invalid ranges), initialize the cost to 0.

3. Fill the Matrices Using Dynamic Programming
   For increasing lengths of subarrays (from 2 to n), calculate the cost and determine the optimal root.

```
for length from 2 to n:
    for i from 0 to n-length:
        j = i + length - 1
        cost[i][j] = infinity
        # Calculate the sum of probabilities for the subarray
        sum_prob = sum(probabilities[k] for k in range(i, j+1))
        # Try making each word k the root and calculate the cost
        for k in range(i, j+1):
            cost_left = cost[i][k-1] if k > i else 0
            cost_right = cost[k+1][j] if k < j else 0
            total_cost = cost_left + cost_right + sum_prob
            if total_cost < cost[i][j]:
                cost[i][j] = total_cost
                root[i][j] = k
```

4. Construct the Optimal BST:
   Use the root matrix to build the BST. Starting from the root of the entire range ($root[0][n-1]$), recursively build the left and right subtrees.

PROBLEM 2

Write pseudocode for the procedure CONSTRUCT-OPTIMAL-BST(root,n) which, given the table root[1:n, 1:n], outputs the structure of an optimal binary search tree. For the example in previous figure, your procedure should print out the

structure:

•K2 is the root

•K1 is the left child of k2

•D0 is the left child of k1

•D1 is the right child of k1

•K5 is the right child of k2

•K4 is the left child of k5

•K3 is the left child of k4

•D2 is the left child of k3

•D3 is the right child of k3

•D4 is the right child of k4

•D5 is the right child of k5

```
PROCEDURE CONSTRUCT-OPTIMAL-BST(root, i, j, parent, direction)
  if i > j
    if direction == "left"
      PRINT "D" + (i-1) + " is the left child of " + parent
    else if direction == "right"
      PRINT "D" + j + " is the right child of " + parent
    return

  k = root[i][j]
  if parent is None
    PRINT "K" + k + " is the root"
  else if direction == "left"
    PRINT "K" + k + " is the left child of K" + parent
  else if direction == "right"
    PRINT "K" + k + " is the right child of K" + parent

  CONSTRUCT-OPTIMAL-BST(root, i, k-1, k, "left")
  CONSTRUCT-OPTIMAL-BST(root, k+1, j, k, "right")

# Main procedure to construct the tree from the root table
PROCEDURE MAIN-CONSTRUCT-OPTIMAL-BST(root, n)
  CONSTRUCT-OPTIMAL-BST(root, 1, n, None, None)
```

PROBLEM 3

Suppose that instead of maintaining the table w[i, j], you computed the value of w[i, j] directly from equation of w(i, j) (slide 10) in line 9 of OPTIMAL-BST and used this computed value in line 11. How would this change affect the asymptotic running time of OPTIMAL-BST?

If we compute the value of $w[i,j]$ directly from its definition in line 9 of the OPTIMAL-BST algorithm instead of maintaining a precomputed table $w[i,j]$, it will significantly affect the asymptotic running time of the algorithm. Here's an analysis of the impact:

Definition of $w(i,j)$

The weight $w[i,j]$ represents the sum of the probabilities of the keys from *ii* to *jj*:

$$w[i,j] = \sum_{k=i}^{j} Pk$$

Original OPTIMAL-BST Algorithm with Precomputed w

1. **Precomputation**: The weights $w[i,j]$ are precomputed in a preprocessing step, which takes $O(n^2)$ time and $O(n^2)$ space.
2. **Dynamic Programming Table Construction**: The algorithm then fills out the cost and root tables in $O(n^3)$ time, assuming the weights are accessed in constant time from the precomputed table.

Modified OPTIMAL-BST Algorithm with On-the-fly Computation of w

If we compute $w[i,j]$ directly using the sum formula whenever it is needed, the computation would be as follows:

1. **Computing $w[i,j]$ on the fly**: Each computation of $w[i,j]$ requires summing $j-i+1$ elements. This takes $O(j-i+1)$ time.
2. **Dynamic Programming Table Construction**: In the worst case, for each subarray defined by i and j, we need to compute the weight:

$$For\ i = 1\ to\ n: For\ j = i\ to\ n: Compute\ w[i,j]\ by\ summing\ pk\ from\ k = i\ to\ j$$

For each pair (i,j), the sum computation takes $O(j-i+1)$ time. The number of pairs (i,j) is $\frac{n(n+1)}{2}$ (the number of entries in the upper triangle of an $n \times n$ matrix). Summing these lengths:

$$\sum_{i=1}^{n}\sum_{j=1}^{n}(j-i+1) = \sum_{i=1}^{n}\left(\sum_{j=1}^{n} j - \sum_{j=1}^{n} i + \sum_{j=1}^{n} 1\right)$$

Simplifying, we get:

$$\sum_{i=1}^{n}\left(\frac{(n-i+1)(n-i+2)}{2}\right) \approx \sum_{i=1}^{n} O((n-i+1)^2)$$

This simplifies to $O(n^3)$.

Thus, the cost of dynamically computing $w[i,j]$ on the fly would add an additional $O(n\text{^}3)$ operations to the algorithm.

Total Running Time

- **With Precomputed Weights**:
    - Precomputation: $O(n^2)$
    - Dynamic Programming: $O(n^3)$
    - **Total**: $O(n^3)$
- **With On-the-fly Weight Computation**:
    - Dynamic Programming: $O(n^3)$ (computing weights) + $O(n^3)$ (filling tables)
    - **Total**: $O(n^3) + O(n^3) = O(n^4)$

Conclusion

If the value of $w[i, j]$ is computed directly from its definition whenever it is needed, the asymptotic running time of the OPTIMAL-BST algorithm would increase from $O(n^3)$ to $O(n^4)$. This is because each computation of $w[i, j]$ would take linear time with respect to the range $j - i + 1$, leading to an additional cubic term in the overall running time. Thus, the algorithm would become significantly less efficient.